UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS	**Universidad Distrital Francisco José de Caldas** **Facultad Tecnológica** **Formato de Inscripción de Trabajo de Grado** **Monografía – Pasantía – Investigación e Innovación** **Tecnología en Sistematización de Datos e Ingeniería en Telemática**	

FORMATO NRO 1. PARA PRESENTACIÓN DE LA PROPUESTA DE TRABAJO DE GRADO V.4			
Carrera:	Ingeniería Telemática por ciclos propedéuticos		
Fecha de elaboración del documento:	22 de mayo de 2025		
Título de trabajo de grado:	**Optimización Integral de Redes Kubernetes: Comparativa de CNIs, Automatización de NetworkPolicies y Reducción de Sobredimensionamientos.**		
Entregable principal de su proyecto de grado: ***(Anexo A: Entregables de un proyecto de grado)***	**Modelo de optimización**		
Nombre ejecutor 1:	Duwan Estiven Sierra Guerrero	**CÓDIGO E.**	**20231678001**
Nombre ejecutor 2:	Holman Audrey Alba Castro	**CÓDIGO E.**	**20231678018**
Correo electrónico ejecutor 1:	DESIERRAG@UDISTRITAL.EDU.CO		
Correo electrónico ejecutor 2:	HAALBAC@UDISTRITAL.EDU.CO		
Teléfono ejecutor 1	3002842881		
Teléfono ejecutor 2	3059264013		
Dirección lindekin ejecutor 1	https://www.linkedin.com/in/duwansierra		
Dirección lindekin ejecutor 2	https://www.linkedin.com/in/holman-alba-castro-866277169		
TIPO DE TRABAJO DE GRADO (Seleccione solo un ítem, marcando con X)			
◉ Monografía	○ Pasantía	○ Investigación e Innovación	

ENTREGABLES DETALLADOS QUE PIENSA INCLUIR O QUE INCLUYÓ EN SU TRABAJO DE GRADO (Seleccione los ítems que usted considere, mínimo 5)			
☐1. Programa inédito	☒2. Diagrama de arquitectura	☒3. Documento técnico	☒4. Documento de arquitectura
☐5. Modelo lógico de datos	☐6. Modelo físico de datos	☐7. Modelo de arquitectura	☐8. Api
☐9. Front	☐10. Middleware	☐11. Backend	☐12. Diseño de red
☐13. Diseño eléctrico	☐14. Planeación estratégica	☐15. Simulador educativo	☐16. Ambiente virtual de aprendizaje
☐17. Objeto virtual de aprendizaje	☐18. Aplicación móvil	☐19. Sistema experto	☐20. Red Neuronal
☐21. Ecuación resuelta	☐22. Ecuación planteada	☐23. Ecuación interpretada	☐24. Cronograma
☐25. Plan de negocios	☐26. Análisis de riesgos	☐27. Implementación de una norma	☐28. Plan de calidad
☐29. Plan de mejoramiento	☐30. Metodología	☒31. Plan de integración	☐32. Plan de seguridad
☐33. Juego	☐34. Aplicación de escritorio	☐35. Aplicación multiplataforma	36. Diseño de navegación
☐37. CRM Custom Relation Management.	☐38. Metodología para implementar norma	☐39. Implementación de tecnología BlockChain	☐40. Implementación de Tecnología IoT.
☐41. Diseño de dispositivos electrónicos	☒42. Estudio de viabilidad	☒42. Modelo de optimización	☐43. Aplicación Web
☐44. Aplicación empotrada	☒45. Modelo arquitectónico	☐46. Sistema de gestión de conocimiento	☐47. Patrón de diseño
☐48. Propuesta de investigación	☐49.Red implementada	☒50. Simulación	☐51. Modelado matemático
☐52. Plan de gestión	☒53. Análisis de un sistema	☐54. Diseño de un sistema	☐55. Estado financiero
☐56. Servicios Cloud	☐57. Sistema telemétrico	☐58. Bodega de datos	☐59. Minería de datos
☐60. Desarrollo de una técnica	☐61. Evaluación de una técnica	☐62. Aplicación de una técnica	☐63. Análisis de requisitos
☐64. Pruebas de usabilidad y/o accesibilidad	☒65. Pruebas de rendimiento y/o seguridad	☐66. Artículo	☒67. Análisis de tecnologías
☐68. Prospectiva tecnológica	☒69. Recomendación de arquitecturas	☒70. Recomendaciones de tecnologías	☐71. Recomendación de metodologías
☐72. Recomendación de dispositivos	☐73. Análisis de eficiencia de un algoritmo	☐74. Recomendación de líneas de investigación	☐75. Recomendación de contenidos temáticos a los syllabus

MODALIDAD DE <u>MONOGRAFÍA</u> (Seleccione solo un ítem, marcando con X)	
- La proposición de una solución a un problema tecnológico mediante la elaboración de modelos, diseños,prototipos y/o productos.	X
- El desarrollo de estudios particulares sobre temas específicos, caracterizado por la ejecución de un proceso de investigación	

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS	**Universidad Distrital Francisco José de Caldas** **Facultad Tecnológica** **Formato de Inscripción de Trabajo de Grado** **Monografía – Pasantía – Investigación e Innovación** **Tecnología en Sistematización de Datos e Ingeniería en Telemática**	

- El desarrollo de un proyecto particular que alimenta un propósito de investigación colectivo, en el marco de una temática definida, tiempo determinado y asistencia obligatoria a un seminario especializado ofrecido por ungrupo de investigación institucionalizado	

INFORMACIÓN DE MONOGRAFÍA			
Director Interno	Gerardo Alberto Castang Montiel.		
Duración estimada del proyecto	9 meses		
Grupo de investigación a la que pertenece propuesta	ORION: Telemática		
Evaluador 1 Sugerido	Darin Jairo Mosquera Palacios	**Evaluador 2 Sugerido**	Elija otro posible jurado.

INFORMACIÓN MODALIDAD DE PASANTIA (Seleccione solo un ítem. Coloque una X)			
☐Investigación	☐Empresarial	☐Contrato de aprendizaje (Solo para Tecnología)	
INFORMACION DE LA PASANTIA			
Nombre de la Empresa /Entidad	Escriba el nombre de la empresa		
Tipo de empresa	Elija el tipo de empresa.	**Teléfono empresa**	Teléfono y celular de la empresa
Persona contacto en empresa	Escriba el nombre completo de la persona	**Teléfono y correo de ese contacto**	Escriba el teléfono y el correo electrónico
Sector al que pertenece la empresa	Sector primario	Elija un elemento.	
	Sector secundario	Elija un elemento.	
	Sector terciario	Elija un elemento.	
	Sector cuaternario	Elija un elemento.	
	Sector quinario	Elija un elemento.	
Dirección física	Escriba la dirección física de la empresa		
Dirección web	Escriba la dirección electrónica de la empresa		

PLANTEAMIENTO DEL PROBLEMA				
Problemas organizacionales ***(Mínimo seleccione 5)***	☒ Oportunidad de transformación tecnológica desaprovechada	☐ Sistemas existentes costosos para la organización	☐Mala la toma de decisiones	☐Mala selección de infraestructura tecnológica
	☐Mala selección de infraestructura de plataforma	☐Mala selección de infraestructura de software	☐Falta de estudios de factibilidad	☒Desaprovechamiento de la infraestructura tecnológica de una organización
	☐Desaprovechamiento del talento humano de una organización	☒Optimización del recurso de infraestructura tecnológica	☐Optimización del recurso de plataforma de una organización	☒Optimización del recurso de software de una organización
	☐Delitos de falsificación	☐Delitos de suplantación	☐Suplantación de identidad	☐Falta de difusión de una información
	☐Falta de sistemas de gestión	☐Falta de sistemas de control	☒Falta de sistemas de monitoreo	☒Falta de sistemas de seguimiento
	☒Falta de sistemas de respaldo	☐Falta de sistemas de autenticación	☐Falta de conectividad	☐Mal uso de una tecnología
	☐Uso subvalorado de una tecnología	☐Uso no conocido de una tecnología	☐Mal aprovechamiento de una infraestructura	☐Mal aprovechamiento de software existente
	☐Amenaza de una tecnología existente	☐Amenaza de una tecnología emergente	☐Evaluación de una tecnología existente	☐Evaluación de una tecnología emergente
	☐Amenaza de una tecnología legada	☐Evaluación de una tecnología legada	☐Insuficiente administración de un riesgo	☐Mala comunicación entre las personas
	☐Falta de documentación de los procesos	☐Mala implementación de una norma	☐Bajo control de un proceso	☒No aprovecha la oportunidad de aplicar tecnología a un sector
Problemas relacionados con los datos ***(Mínimo seleccione 5)***	☐Falta de confidencialidad en el manejo de la información	☐Falta de precisión, en el manejo de los datos	☐Falta de consistencia en el manejo de los datos	☐Falta de confiabilidad en el manejo de los datos
	☐Falta de acceso a la información y a los datos	☒Falta de disponibilidad de los sistemas que manejan datos	☐Dificultad para almacenar los datos	☐Dificultad para recuperar los datos
	☒Dificultad para visualizar los datos	☐Dificultad para integrar los datos	☐Dificultad para adaptar los datos	☐Dificultad para limpiar los datos
	☐Dificultar para almacenar los datos	☐Dificultad para interpretar los datos	☐Dificultad para proyectar los datos	☒Dificultad para analizar los datos
	☐Dificultad para extraer los datos	☐Dificultad para transformar los datos	☐Dificultad para extraer los datos de fuentes heterogéneas	☐Dificultad para convertir los datos

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS	**Universidad Distrital Francisco José de Caldas** **Facultad Tecnológica** **Formato de Inscripción de Trabajo de Grado** **Monografía – Pasantía – Investigación e Innovación** **Tecnología en Sistematización de Datos e Ingeniería en Telemática**	

	☐Dificultad para sistematizar los datos	☐Dificultad para ordenar los datos	☒Dificultad para transportar los datos	☒Dificultad para asegurar la calidad de los datos
	☐Dificultad para integrar los datos	☐Dificultad para consolidar los datos	☒Dificultad para garantizar la integridad de los datos	☐Dificultad para relacionar los datos
	☐Dificultad para distribuir los datos	☐Dificultad para sincronizar los datos	☐Dificultad para replicar los datos	☐Dificultad para respaldar los datos
	☐Dificultad para ofuscar los datos	☐Dificultad para reproducir los datos	☐Dificultad para seleccionar los datos	☐Dificultar para comparar los datos
	☐Dificultad para enumerar los datos	☐Dificultad para convertir los datos	☐Dificultad para estimar los datos	☐Dificultad para inferir los datos
	☐Dificultad para relacionar los datos	☐Dificultad para traducir los datos	☐Dificultad para preparar los datos	☐Dificultad para calcular los datos
	☐Dificultad para construir los datos	☐Dificultad para agregar más datos	☐Dificultad para descomponer los datos	☐Dificultad para dividir los datos
	☐Dificultad para discriminar los datos	☐Dificultad para catalogar los datos	☐Dificultad para inferir los datos	☐Dificultad para combinar los datos
	☒Dificultad para anonimizar los datos	☐Dificultad para normalizar los datos	☐Dificultad para crear los datos	☐Dificultad para reconstruir los datos
	☐Dificultad para revisar los datos	☐Dificultad para evaluar los datos	☐Dificultad para valorar los datos	☐Dificultad para eliminar los datos
	☐Dificultad para actualizar los datos	☐Dificultad para interpretar los datos	☐Dificultad para agrupar los datos	☐Dificultad para segmentar los datos
	☒Dificultad para garantizar la confidencialidad de los datos	☐Dificultad para modelar los datos	☒Dificultad para autenticar los datos	☐Dificultad para identificar los datos

Problemas ingenieriles ***(Mínimo seleccione 1)***	**PROBLEMAS DE FUNCIONALIDAD**	
	☐Problemas de exactitud	Dificultad para que el sistema procese los datos con la exactitud que se requiere
	☐Problemas de interoperabilidad	Dificultad para que el sistema interactúe con otro sistema
	☒Baja seguridad del sistema	Medida en que los datos están desprotegidos
	☒Problemas de acceso no autorizado	Personas o sistemas no autorizados acceden a sistema y realizan acciones no permitidas
	☐Bajo uso de estándares	No se acatan estándares aprobados en el área del proyecto
	☐Bajo cumplimiento de regulaciones	No se cumplen muchas regulaciones legales o especificaciones técnicas
	☐Problemas de conformidad	El sistema no funciona de acuerdo con referentes de la funcionalidad que debería tener
	PROBLEMAS DE EFICIENCIA	
	☒Alta latencia	Está relacionado con la eficiencia. El tiempo dedicado a responder a un evento es alto
	☐Consume muchos recursos	Está relacionado con la eficiencia
	☐Baja escalabilidad	La escalabilidad es la capacidad del sistema para manejar aumentos de carga sin disminuir el rendimiento, o la posibilidad de aumentar rápidamente la carga
	PROBLEMAS DE FIABILIDAD	
	☐Falta de tolerancia a fallos	Poca capacidad de mantener un nivel especificado de funcionamiento en caso de errores del software
	☐Falta de capacidad de recuperación	Baja capacidad del sistema para que este se reestablezca con datos en caso de presentarse una falla
	☒Falta de madurez del sistema	Madurez es la capacidad para evitar fallas como resultado de errores en el sistema
	☐Falta de conformidad de la fiabilidad	Dificultad del sistema de adherirse a normas o regulaciones relativas a la fiabilidad
	☐Baja disponibilidad del sistema	Dificultad para que el sistema funcione en un punto dado de tiempo
	PROBLEMAS DE MANTENIBILIDAD	
	☐Dificultad para diseñar un sistema	Dificultad para extraer la arquitectura del sistema
	☐Dificultad de cambiabilidad	Dificulta para modificar un subsistema o un componente
	☒Dificultad para analizar el sistema	Dificultad para generar diagnósticos de deficiencias o identificar partes dañadas
	☒Dificultad para evaluar el sistema	Dificultad para identificar componentes o subsistemas dañados o problémicos
	☐Baja estabilidad del sistema	Se presentan efectos inesperados debido a modificaciones del sistema
	☐Baja extensibilidad	Dificultad para que al sistema se le añadan nuevas funcionalidades
	☐Baja reusabilidad	Baja posibilidad de reusar el sistema por parte de otro sistema con mínimos cambios y configuraciones
	PROBLEMAS DE USABILIDAD	
	☐Dificultad de comprender del sistema	Dificultad para que los usuarios entiendan el funcionamiento del sistema
	☐Dificultad de aprendizaje	Dificultad del sistema para permitir que el usuario aprenda
	☐Dificultad de operabilidad	Dificultad del sistema para permitir que el usuario lo opere
	☐Dificultad de atracción	Poca atractividad del usuario por el sistema
	PROBLEMAS DE PORTABILIDAD	
	☐Problemas de adaptabilidad	Problemas para que el sistema sea adaptado a otro entorno
	☐Problemas de instalación	Dificultades para que un sistema sea instalado en un ambiente especificado

	☒Problemas de coexistencia	Dificultad para que el sistema coexista con otros sistemas compartiendo recursos
	☐Problemas de reemplazabilidad	Dificultad para que el sistema sea usado en lugar de otro para el mismo propósito y en el mismo entorno
	☐Dificultad de escalabilidad	Dificultades para para que crezca un sistema en cuanto a recursos, usuarios o carga

DEFINICIÓN Y PLANTEAMIENTO DEL PROBLEMA (Observar Anexo B)	
Definición del problema	En la actualidad, las organizaciones han adoptado masivamente Kubernetes como plataforma para ejecutar sus aplicaciones empresariales utilizando contenedores y microservicios. Esta tecnología promete facilitar el despliegue, la escalabilidad y la disponibilidad de sistemas críticos para el negocio. Sin embargo, a medida que más empresas implementan Kubernetes en sus entornos de producción, surge un problema recurrente y complejo: la gestión efectiva de la red del clúster. Este problema no es menor, ya que la red es el componente que permite que todos los servicios y aplicaciones se comuniquen entre sí de manera segura y eficiente. Cuando la red no funciona correctamente, las aplicaciones fallan, los servicios se vuelven lentos y la experiencia del usuario final se ve comprometida. La magnitud de este problema ha crecido proporcionalmente con la adopción de Kubernetes, afectando desde pequeñas startups hasta grandes corporaciones multinacionales que dependen de estos sistemas para sus operaciones diarias [2]. El origen de esta problemática radica en que Kubernetes, aunque es una plataforma poderosa, no resuelve automáticamente todos los aspectos de la comunicación de red entre aplicaciones. En su lugar, delega estas funciones a componentes adicionales llamados CNI (Container Network Interface) [4], que son como "plugins" de red que cada organización debe seleccionar e instalar. Existen múltiples opciones disponibles como Calico, Cilium, Antrea y Flannel, cada una con diferentes características, ventajas y desventajas. El problema surge cuando las organizaciones deben elegir entre estas opciones sin contar con criterios claros, comparaciones objetivas o modelos específicos para evaluar cuál se adapta mejor a sus necesidades específicas[7]. Esta falta de orientación lleva a decisiones basadas en recomendaciones informales, preferencias personales o información parcial, lo que posteriormente resulta en problemas de rendimiento, costos elevados y dificultades operativas que son costosas y complejas de corregir una vez que el sistema está en producción. Uno de los aspectos más críticos de este problema es la seguridad de la red dentro del clúster de Kubernetes. Por defecto, Kubernetes permite que cualquier aplicación se comunique libremente con cualquier otra aplicación dentro del clúster, lo cual representa un riesgo significativo de seguridad. Para controlar esta comunicación, existen mecanismos llamados "políticas de red" que actúan como reglas de firewall internas, pero implementar estas políticas correctamente es extremadamente complejo. Los equipos de desarrollo y operaciones deben definir qué servicios pueden comunicarse entre sí, desde qué puertos, bajo qué condiciones y en qué momentos [10]. Esta tarea se vuelve exponencialmente más difícil cuando se tienen decenas o cientos de aplicaciones funcionando simultáneamente, múltiples equipos de desarrollo trabajando de forma independiente, y diferentes entornos como desarrollo, pruebas y producción que requieren configuraciones distintas. Los errores en estas configuraciones pueden resultar en brechas de seguridad que expongan información sensible o permitan accesos no autorizados, pero también pueden bloquear comunicaciones legítimas causando interrupciones del servicio. Otro aspecto fundamental del problema es la dificultad para obtener visibilidad clara sobre lo que realmente está ocurriendo en la red del clúster. No es suficiente con configurar políticas de red y asumir que funcionan correctamente; las organizaciones necesitan monitorear constantemente el tráfico de red, verificar que las reglas se estén aplicando como se esperaba, detectar comportamientos anómalos e identificar posibles problemas antes de que afecten a los usuarios finales. Sin embargo, las herramientas tradicionales de monitoreo de red no están diseñadas para el entorno dinámico y complejo de Kubernetes, donde los servicios aparecen y desaparecen constantemente, las direcciones IP cambian frecuentemente y el tráfico fluye a través de múltiples capas de “extracción”. Esta falta de visibilidad deja a los equipos técnicos "a ciegas", dependiendo de alertas básicas o reportes manuales que llegan demasiado tarde para prevenir incidentes[6]. Como resultado, muchas organizaciones operan sus sistemas con un nivel de incertidumbre inaceptable sobre el estado real de su infraestructura de red. La combinación de todos estos desafíos resulta en un impacto significativo en los costos operacionales y la eficiencia general de las organizaciones. Ante la incertidumbre sobre el rendimiento y la confiabilidad de la red, muchos equipos adoptan un enfoque conservador, asignando más recursos de los necesarios (CPU, memoria, ancho de banda) como medida de seguridad para evitar interrupciones del servicio. Esta práctica, conocida como sobredimensionamiento, genera desperdicio de recursos y costos innecesarios que pueden representar miles o millones de dólares anuales dependiendo del tamaño de la organización. Además, sin visibilidad clara sobre el comportamiento real del sistema, es imposible optimizar el uso de recursos de manera inteligente o implementar mecanismos de escalado automático confiables [5]. Los equipos técnicos terminan dedicando tiempo considerable a resolver problemas de red de forma reactiva en lugar de enfocar sus esfuerzos en desarrollar nuevas funcionalidades o mejorar la experiencia del usuario. Esta situación no solo afecta la competitividad de las organizaciones, sino que también genera frustración en los equipos técnicos [10] y puede impactar negativamente los acuerdos de nivel de servicio con clientes internos y externos.
Planteamiento del problema	**Árbol de problemas**

Problema central: Implementación y selección inadecuada de soluciones CNI en clústeres Kubernetes, lo cual degrada el rendimiento, expone vulnerabilidades de seguridad y genera ineficiencias operativas (costos y consumo energético).

Causas Directas:

CD1: Falta de criterios: Las empresas no dispone de una estandarización al momento de seleccionar o comparar distintos plugins CNI, sin estos estándares, la elección suele basarse en lo que recomienda internet o en pruebas rápidas no técnicas que no reflejan la realidad.

CD2: Brecha de competencias: Complejidades técnicas propias de los CNIs sencillos y avanzados, esto desde el término técnico como en la escasez de talento humano con conocimiento especializado, esto brinda configuraciones erroneas, dependencias de terceros y más tiempo para implementar en ambientes productivos.

CD3: Políticas Insuficientes o mal implementadas: Implementaciones deficientes e incompletas de las Network Policies que brinda Kubernetes por defecto, donde se dejan brechas de seguridad importantes y se dificulta el mantenimiento de estas.

CD4: Arquitectura abierta: Modelo de red abierto ya que la comunicación entre los pods es por defecto "abierta" ya que desde el inicio estos pods se hablan sin discriminarse.

CD5: Falta de Visibilidad: No se tiene una herramienta que nos brinde con claridad que ocurre con el tráfico de red entre los pods. Sin esta información, si tenemos un problema solamente lo descubriremos cuando este afecte al usuario.

CD6: Sobredimensionamiento de los recursos: Al no conocer cuál es la carga real para cada Pod o CNI, las empresas compran servidores o servicios de nube con la premisa del "por si acaso" gastando dinero y energía en recursos que no se utilizan.

Causas subyacentes

CS1-CD1: Falta de lineamientos internos: No existen guías corporativas que definan que métricas de red, seguridad y consumo deben ser tenidas en cuenta antes de elegir un CNI lo cual deja la decisión a la intuición del equipo.

CS2-CD1: Documentación dispersa: La información técnica sobre los CNIs se encuentra repartida en blogs, documentaciones oficiales demasiado especializadas, donde no hay un comparativo oficial consolidado y fácil de consultar que nos ayude a tomar las decisiones.

CS3-CD2: Escases de capacitaciones: Los cursos disponibles cubren implementaciones con Kubernetes básicas sin agregar el apartado de redes, por lo tanto, el personal no adquiere habilidades para desplegar ni mantener CNIs de red complejos.

CS4-CD2: Alto personal en rotación: Los ingenieros que adquieren algo de experiencia con los CNIs propuestos tienden a cambiar de empleo rápidamente, por lo tanto, el conocimiento se va con ellos y no hay una documentación fuerte sobre el qué y cómo se implementó.

CS5-CD3: Copias de ejemplos sin validar: Muchos equipos copian y pegan políticas de red encontradas en repositorios públicos o brindados por IAs sin adaptarlas ni probarlas en escenarios reales ni ambientes de pruebas.

CS6-CD3: Falta de pruebas automatizadas: No existen pipelines ni softwares disponibles que nos confirmen que las reglas de red siguen siendo efectivas en los bloqueos cuando los CNIs o Clúster cambiaron o simplemente se actualizaron.

CS7-CD4: Cultura de "todo abierto funciona": Se prioriza que los clusters arranquen rápidamente y de una forma "sencilla" dejando puertos y rutas primordiales abiertas, dejando la seguridad recaída al momento de la primera implementación, dejando este endurecimiento de la red para un futuro indeterminado.

CS8-CD5: Herramienta de monitoreo ilimitadas: Las plataformas instaladas solo nos brindan información básica de las herramientas (CPU/RAM...) siendo el tráfico de red un extra en licencias profesionales que no se adquieren.

CS9-CD5: Falta de alertas útiles: No se generan alarmas que indiquen picos de tráfico inusual, actividades sospechosas, pods con comportamientos no contemplados, lo que retrasa la reacción ante cualquier incidente.

CS10-CD6: Presupuestos mal estimados: Planeación financiera sin pruebas de carga contempladas, mediciones de consumo ineficientes, cultura de "si funciona para que cambiarlo", y el adquirir infraestructura "de más por si acaso" para evitar alguna sorpresa.

CS11-CD6: Política de compras rígidas: Los procesos internos obligan a adquirir hardware anualmente y en grandes lotes o promociones, impidiendo ajustar recursos según la demanda real.

La operación de redes en Kubernetes plantea desafíos que van más allá de simplemente hacer funcionar los contenedores de aplicaciones. La elección y configuración del CNI y la definición de reglas de comunicación entre servicios tienen un impacto directo en la estabilidad, el rendimiento y la seguridad del sistema completo. Además, estas decisiones influyen significativamente en los costos operativos de las organizaciones. Aunque Kubernetes proporciona un modelo de red base común, delega las decisiones críticas de implementación en estos complementos especializados, los cuales difieren considerablemente en sus capacidades y comportamientos cuando se enfrentan a cargas de trabajo reales[4]. En consecuencia, las organizaciones se encuentran en la difícil posición de seleccionar y operar estas tecnologías sin contar con un marco práctico, claro y comparable que les permita tomar decisiones fundamentadas en evidencia y mantener el control efectivo del tráfico a lo largo del tiempo[7].

El análisis mediante árbol de problemas revela como problema central la implementación y selección inadecuada de soluciones de red, lo cual genera una cascada de efectos negativos que degradan el rendimiento del sistema, exponen vulnerabilidades críticas de seguridad y producen ineficiencias operacionales significativas. Esta problemática se manifiesta porque la diversidad de opciones disponibles y sus respectivas configuraciones condicionan directamente aspectos fundamentales como la latencia de respuesta, el rendimiento general del sistema y el uso eficiente de recursos de procesamiento[9]. Sin embargo, estas diferencias cruciales frecuentemente no son visibles durante las pruebas preliminares o evaluaciones superficiales que realizan los equipos técnicos.

La situación se agrava considerablemente cuando las organizaciones no implementan políticas de red estrictas desde el inicio, o cuando heredan configuraciones de otros proyectos sin verificar adecuadamente su funcionamiento en los nuevos contextos de producción. Esta realidad evidencia la necesidad urgente de un planteamiento sistemático que delimite con precisión qué elementos comparar, cómo realizar estas comparaciones de manera objetiva, y cómo mantener una segmentación de red efectiva sin elevar innecesariamente la complejidad operativa.

La primera causa directa identificada es la ausencia de criterios estandarizados para seleccionar y comparar las diferentes CNI disponibles. Esta carencia de estándares mínimos y escenarios reproducibles provoca que las decisiones organizacionales se apoyen únicamente en documentación dispersa, recomendaciones informales de terceros o experiencias anecdóticas de otros equipos. Aunque la especificación técnica oficial describe responsabilidades y contratos básicos, no define procedimientos comparativos operativos entre las diferentes soluciones disponibles [18]. Diversos estudios de rendimiento han demostrado diferencias relevantes y medibles entre soluciones y topologías de red, lo que refuerza la necesidad crítica de contar con un conjunto de indicadores técnicos simples y aplicables. Estos indicadores deberían incluir métricas como latencia , rendimiento de transferencia llamado throughput, carga de procesamiento del plano de datos y estabilidad del sistema bajo cambios operativos [8]. Las subcausas que alimentan este problema incluyen lineamientos internos insuficientes dentro de las organizaciones y la falta de comparativos consolidados y accesibles, situación que traslada inevitablemente las decisiones técnicas críticas al nivel de intuición y experiencia personal de los equipos de trabajo.

La segunda causa directa corresponde a la brecha significativa de competencias técnicas especializadas en redes de Kubernetes que existe en la mayoría de organizaciones. Muchos equipos técnicos dominan efectivamente el despliegue básico de contenedores y aplicaciones, pero carecen de formación específica y profunda en aspectos como segmentación de red, diferentes modos de servicio, o las particularidades técnicas de cada solución de CNI disponible [16]. Esta situación se ve agravada por los altos índices de rotación de personal técnico especializado, lo cual dificulta enormemente la preservación del conocimiento adquirido y la documentación de procedimientos críticos. Estas condiciones organizacionales favorecen inevitablemente la implementación de configuraciones incompletas o subóptimas, generan dependencia excesiva de proveedores externos o consultores, y extienden significativamente los ciclos de implementación en ambientes productivos. Los reportes especializados del ecosistema tecnológico y las guías de mejores prácticas coinciden consistentemente en señalar que los errores de configuración inicial y la escasez de habilidades especializadas constituyen factores recurrentes de riesgo operativo y fuentes importantes de ineficiencia en las organizaciones. Las subcausas específicas identificadas incluyen programas de capacitación que se centran únicamente en aspectos básicos de la tecnología y la pérdida sistemática de conocimiento especializado debido a la alta rotación del personal técnico.

La tercera causa directa se relaciona directamente con la implementación insuficiente o técnicamente inadecuada de las políticas de red que controlan la comunicación entre servicios. Cuando las organizaciones no implementan políticas de red, la interacción entre componentes del sistema tiende a permanecer completamente abierta por defecto [2], lo cual facilita los primeros pasos de implementación, pero resulta completamente inadecuado para entornos con requerimientos estrictos de control de acceso. Por otro lado, cuando estas políticas sí se implementan, su redacción técnica, mantenimiento continuo y evolución resultan procesos extremadamente complejos y altamente propensos a errores humanos que pueden tener consecuencias graves [10]. El análisis identifica subcausas específicas como la práctica común de copiar ejemplos de configuración desde repositorios públicos sin adaptarlos ni probarlos adecuadamente, y la ausencia total de pruebas automatizadas que verifiquen continuamente que las reglas de comunicación siguen siendo efectivas después de cambios en el sistema o actualizaciones de los componentes de red. Los

marcos de referencia modernos de seguridad insisten en la necesidad de "microsegmentación" y verificación continua del tráfico, pero su implementación práctica en Kubernetes requiere guías procedimentales detalladas y validaciones periódicas sistematizadas para evitar efectivamente la existencia de permisos excesivos y puntos ciegos de seguridad.

La cuarta causa directa corresponde a la arquitectura abierta por defecto que caracteriza a Kubernetes, la cual prioriza la conectividad inicial y la facilidad de arranque, pero puede dejar rutas de comunicación innecesarias habilitadas si no se implementa un proceso de endurecimiento desde las primeras etapas de implementación. Esta filosofía de diseño que permite iniciar con "todo abierto" para facilitar la puesta en marcha [11], postergando las configuraciones restrictivas para etapas posteriores, se registra en el análisis como una cultura organizacional que normaliza y acepta riesgos evitables como parte del proceso normal de operación. Si esta situación no se corrige implementando criterios de seguridad estrictos y procesos de verificación continua desde el inicio, la superficie de exposición a amenazas crece exponencialmente y la organización desarrolla una falsa sensación de seguridad basada en supuestos incorrectos en lugar de fundamentarse en evidencia verificable y medible. Esta causa se manifiesta a través de la subcausa cultural identificada como "todo abierto funciona", donde se prioriza que los sistemas arranquen rápidamente y de forma aparentemente sencilla, relegando la implementación de medidas de seguridad a un futuro indeterminado que frecuentemente nunca llega.

La quinta causa directa identificada es la falta crítica de visibilidad útil y accionable sobre el tráfico de red que realmente ocurre dentro del sistema. Muchas plataformas de monitoreo proporcionan indicadores básicos de procesamiento y memoria, pero dejan la observabilidad especializada de red como una capacidad limitada o sujeta a licencias comerciales adicionales costosas. Sin telemetría de flujo adecuada y sin alertas específicas sobre comportamientos anómalos en la comunicación entre servicios, los equipos técnicos descubren los problemas únicamente cuando ya están afectando directamente a los usuarios finales y el impacto en el negocio es evidente [6]. El análisis específica como subcausas la limitación de las herramientas de monitoreo instaladas, que solo brindan información básica sobre recursos computacionales, siendo el análisis detallado del tráfico de red considerado un complemento premium, y la ausencia de alertas verdaderamente accionables que permitan identificar picos de tráfico inusual, actividades sospechosas o comportamientos no contemplados en el diseño original. La mejora de la visibilidad operativa, idealmente implementando inspección de tráfico en múltiples niveles de la arquitectura, resulta crucial para confirmar que las políticas de seguridad funcionan exactamente como se diseñaron y para detectar desviaciones o problemas potenciales con tiempo suficiente para implementar acciones correctivas efectivas [23].

La sexta causa directa corresponde al sobredimensionamiento sistemático de recursos (computacionales que implementan las organizaciones como respuesta a la incertidumbre sobre la demanda real del sistema y los efectos específicos que la configuración de red tiene en el desempeño de los servicios. Ante esta incertidumbre técnica y operativa, resulta una práctica común establecer márgenes amplios de seguridad en las asignaciones de recursos y adquirir capacidad adicional "por precaución", lo cual conduce directamente a subutilización significativa de la infraestructura y costos operativos elevados sin beneficios proporcionales en rendimiento o disponibilidad. Aunque existen mecanismos de escalado automático que pueden contribuir a resolver este problema, su implementación efectiva requiere que se alimenten de señales confiables y precisas, además de una comprensión profunda de las interacciones complejas entre el plano de red y los diferentes modos de operación del sistema de distribución de servicios. El análisis asocia esta causa con subcausas específicas como la planificación presupuestaria sin incluir pruebas de carga representativas y políticas de compra organizacional rígidas que impiden ajustar la capacidad según la demanda real medida, obligando a adquirir infraestructura en lotes anuales que no corresponden necesariamente con las necesidades técnicas reales del sistema.

A partir de este diagnóstico integral, surge la pregunta central que orienta esta investigación: ¿Cómo pueden las organizaciones comparar de manera objetiva las diferentes tecnologías de red disponibles para Kubernetes y automatizar la implementación y verificación de políticas de comunicación segura, logrando reducir riesgos operativos y evitar el desperdicio de recursos sin comprometer el rendimiento del sistema? Esta pregunta principal se desarrolla a través de cuatro interrogantes específicos que guían el trabajo de investigación. Primero, se busca identificar qué métricas fundamentales permiten realizar comparaciones válidas entre tecnologías de red en escenarios que representen fielmente las condiciones empresariales reales. Segundo, se investiga cómo transformar estas métricas técnicas en criterios de selección comprensibles y aplicables para equipos que no necesariamente cuentan con especialización profunda en redes. Tercero, se explora qué tipos de verificaciones automatizadas pueden confirmar de manera continua que las políticas de comunicación siguen cumpliendo efectivamente su propósito de protección cuando el sistema evoluciona o se actualiza. Finalmente, se analiza qué indicadores específicos de monitoreo de red proporcionan la información necesaria para ajustar la capacidad del sistema y reducir el exceso de recursos sin afectar negativamente la calidad del servicio ofrecido a los usuarios finales.

Por último, reconocemos que nos encontramos ante un desafío que combina aspectos técnicos y organizacionales de gran complejidad. Las organizaciones que seleccionan y operan infraestructuras de red en Kubernetes sin contar con criterios comparables objetivos ni procesos de validación continua inevitablemente toman decisiones que incrementan los riesgos de seguridad, degradan el rendimiento del sistema y elevan significativamente los costos operacionales. La solución a esta problemática requiere un enfoque integral que permita realizar comparaciones fundamentadas en datos obtenidos bajo contextos realistas de operación, implementar y mantener comunicaciones seguras mediante políticas técnicamente sólidas y continuamente verificadas, y establecer conexiones efectivas entre la observabilidad del sistema y las decisiones relacionadas

	con la asignación de capacidad computacional. Esta orientación metodológica mantiene coherencia absoluta con la definición inicial del problema y dirige la investigación hacia la obtención de respuestas medibles, reproducibles y directamente aplicables en entornos organizacionales que, sin requerir equipos altamente especializados en tecnologías de red, necesitan operar sus sistemas de manera segura, eficiente y económicamente viable a largo plazo.

FICHA TÉCNICA SI EL PROYECTO INVOLUCRA DESARROLLO DE SOFTWARE			
Lenguajes usados	Enúncielos		
Metodologías usadas	Enúncielas		
Tipo de programación *(Revisar ANEXO F: TIPOS DE PROGRAMACION)*	Programación dirigida por eventos		
Entorno de desarrollo			
Elementos de la misión del programa académico con los que se identifica el proyecto de grado *(Mínimo 3 elementos)*			
Elementos de la misión de la universidad distrital con los que se identifica el proyecto de grado *(Mínimo 3 elementos)*			

OBJETIVO GENERAL (Ver anexo C y D)

Desarrollar un modelo de evaluación, selección y validación de plugins CNI en Kubernetes, compuesto por una guía metodológica, un prototipo de pruebas de laboratorio y una lista de verificación.

OBJETIVOS ESPECÍFICOS (Ver anexo C y D)

1. Evaluar cuantitativamente el rendimiento de los CNI disponibles para Kubernetes (Calico, Cilium, Flannel y Antrea) mediante pruebas controladas de latencia, velocidad de transferencia y uso de recursos, para identificar sus fortalezas y limitaciones en escenarios reales.

2. Evaluar cómo la configuración segura y automatizada de las políticas de red adaptadas a cada plugin CNI reduce la superficie de ataque y la probabilidad de incidentes de seguridad en clústeres Kubernetes, con el fin de determinar su capacidad de reducir vulnerabilidades y prevenir incidentes de seguridad en entornos de laboratorio representativos de producción..

3. Desarrollar, como parte del modelo propuesto, un conjunto de criterios y métricas que permitan la comparación objetiva entre diferentes CNI para Kubernetes mediante la definición de umbrales de rendimiento y factores de ponderación, para facilitar la toma de decisiones técnicas basadas en evidencia cuantificable.

4. Implementar un prototipo funcional que recomiende y configure el CNI más adecuado según las necesidades específicas de cada organización, para simplificar el proceso de implementación y apoyar la definición de configuraciones recomendadas en entornos de prueba.

Mapeo de objetivos con tecnologías usadas para implementar este objetivo	**ANEXO 4**

PREGUNTAS (VER ANEXO C y D)

- ¿Cuáles son las diferencias en velocidad de respuesta entre los CNI para Kubernetes (Calico, Cilium, Flannel y Antrea) cuando las aplicaciones se ejecutan en el mismo servidor versus servidores diferentes bajo condiciones de carga empresarial típica?

- ¿Por qué la elección y configuración del CNI es un factor clave para la seguridad y el rendimiento de un clúster de Kubernetes?
- ¿Cómo impacta cada CNI en el consumo energético del sistema cuando se procesan grandes volúmenes de solicitudes y se ajustan automáticamente los recursos computacionales según la demanda?
- ¿Qué tan efectivo es cada CNI para proporcionar visibilidad del tráfico de datos que permita detectar tempranamente patrones de uso anómalos o picos de actividad inesperados?
- ¿En qué medida se puede reducir el desperdicio de recursos computacionales (procesamiento y memoria) aplicando estrategias de dimensionamiento inteligente basadas en las características específicas de cada CNI?

LLENE LOS SIGUIENTES DATOS PARA CUALQUIER MODALIDAD DE TRABAJO DE GRADO:

FUNDAMENTACIÓN DEL PROYECTO (Para todas las modalidades)	
Planteamiento del Problema	La adopción de Kubernetes como plataforma de orquestación de contenedores ha crecido de manera acelerada en empresas de distintos tamaños, convirtiéndose en un estándar de facto para el despliegue de aplicaciones modernas. Sin embargo, el éxito de esta tecnología depende en gran medida del plano de red, ya que de él dependen la conectividad, el rendimiento, la seguridad y la eficiencia de los servicios. Kubernetes delega esta responsabilidad en los complementos de red (CNI), los cuales ofrecen múltiples alternativas con diferentes capacidades y costos operativos. El reto para las organizaciones es que no existe un modelo práctico y claramente estructurado que permita seleccionar, implementar y mantener estas soluciones de manera objetiva. Las decisiones suelen basarse en pruebas limitadas, en documentación dispersa o en experiencias anecdóticas, lo que conduce a elecciones que no reflejan las condiciones reales de operación. En consecuencia, se generan configuraciones inseguras, políticas de red incompletas y sobredimensionamiento de recursos, problemas que impactan directamente en el rendimiento de los clústeres y en los costos de operación. A nivel operativo, la falta de criterios de comparación dificulta la selección adecuada del CNI; la escasez de competencias especializadas retrasa implementaciones y aumenta la dependencia de terceros; y la ausencia de observabilidad sobre el tráfico de red provoca que los fallos se identifiquen únicamente cuando afectan al usuario final. Estos vacíos no solo degradan la calidad del servicio, sino que incrementan la exposición a riesgos de seguridad y generan gastos innecesarios en infraestructura y consumo energético. La problemática se intensifica porque, aunque Kubernetes provee mecanismos como las NetworkPolicies o el autoescalado de recursos, su aplicación práctica requiere conocimientos avanzados y pruebas sistemáticas que la mayoría de equipos no realiza. En entornos donde los tiempos de entrega y la disponibilidad son críticos, operar bajo supuestos en lugar de evidencias convierte la red en un punto débil. En este contexto, surge la necesidad de investigar cómo comparar de forma objetiva distintos CNIs y cómo automatizar la validación de políticas de red, de modo que las organizaciones puedan garantizar seguridad, rendimiento y eficiencia sin incrementar la complejidad de la operación. Resolver esta brecha es clave para que Kubernetes cumpla su promesa de escalabilidad y confiabilidad, y para que las empresas reduzcan riesgos y costos asociados a la infraestructura tecnológica.
Marco Teórico (Estado del arte)	**K8s Networking: ¿cómo se comunican las cosas dentro y fuera del clúster?**

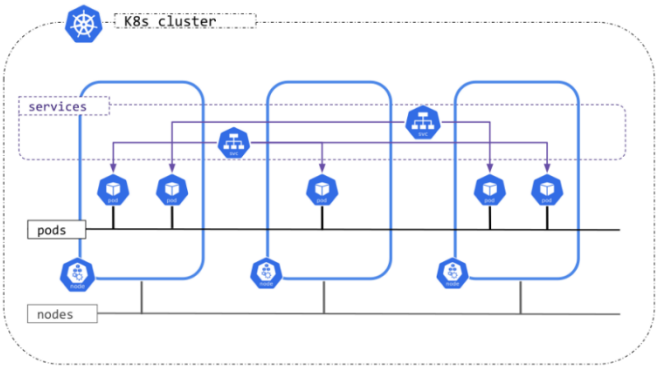


Figura 1. Topología de red de un clúster de Kubernetes (adaptado de [2]).

Mapa mental de redes en Kubernetes (analogía de ciudad)

Un clúster puede conceptualizarse como una ciudad organizada: los *Pods* son las "viviendas" donde se ejecuta la aplicación; los *Nodes* agrupan múltiples viviendas y proveen recursos comunes; los *Services* operan como una central con un número estable para localizar réplicas; y *Ingress/Gateway* implementan el acceso controlado desde el exterior (Internet), actuando como portería y punto de gobierno del tráfico.

Pods = casas (identidad y direcciones)

Kubernetes adopta el modelo *IP-por-Pod*: cada Pod obtiene su propia dirección IP, habilitando comunicación directa entre cargas sin traductores intermedios. Dado que la identidad del Pod es efímera —las réplicas se crean, destruyen y sustituyen de manera dinámica— la plataforma requiere abstracciones estables para descubrimiento y direccionamiento consistentes hacia los destinos correctos [2].

Nodes = barrios (agrupación y recursos comunes)

Un Node hospeda múltiples Pods y suministra CPU, memoria y conectividad. En este nivel se sostienen las rutas y los mecanismos de reenvío que aseguran la entrega de paquetes al Pod adecuado, conforme al modelo de red del clúster (alcance entre Pods, no superposición de rangos, conectividad *east–west*) descrito por Kubernetes [2].

Services = central telefónica (número estable y reparto)

Un Service expone un identificador estable (p. ej., *ClusterIP* o IP virtual) que funciona como "número de central" para un conjunto de Pods equivalentes. El cliente se conecta a ese número y la plataforma balancea hacia una de las réplicas disponibles, desacoplando a los consumidores de los cambios en el ciclo de vida de los Pods [3], [13]. Para mantener actualizada la relación de destinos, Kubernetes emplea *EndpointSlices*, que enumeran direcciones y puertos de los Pods que respaldan un Service, mejorando la escalabilidad y reduciendo la sobrecarga de control frente a los *Endpoints* tradicionales [1].

Ingress / Gateway = portería del conjunto (entrada ordenada desde la calle)

El tráfico externo no accede directamente a los Pods: ingresa mediante una capa de entrada. Con *Gateway API* se formalizan roles (infraestructura, operador, equipo de aplicación) y reglas L7 (host, rutas, TLS) para decidir por qué puerta se entra y a qué Service debe encaminarse cada solicitud [17]. Esta capa organiza el ingreso HTTP/HTTPS y refuerza la separación de responsabilidades sin exponer puertos arbitrarios.

Calles y cableado: CNI (cómo se conectan barrios y casas)

La malla vial la provee *Container Network Interface* (CNI): un conjunto de plugins intercambiables que conectan Pods y Nodes, asignan IPs y programan rutas. Kubernetes requiere un plugin compatible para la conectividad básica; la especificación CNI define cómo el *runtime* solicita "conectar una vivienda" (interfaces, IP y rutas) y cómo liberar recursos al eliminarla [4], [18]. A modo ilustrativo, *host-local* (asignación local de direcciones) y *port-map* (traducción de puertos con el host) muestran que CNI cubre identidad y conectividad de red, no funciones avanzadas de capa 7 [20], [21].

La autopista previa a la portería: balanceo de carga externo

En numerosos despliegues, el tráfico desde Internet hacia Ingress/Gateway atraviesa primero un balanceador de carga externo L3/L4. Diseños de alto rendimiento como *Maglev* evidencian cómo el *hash* consistente y el seguimiento de flujos contribuyen

a una distribución estable y resiliente antes de alcanzar el clúster [5]. No implica que Kubernetes "use Maglev por defecto", sino que ilustra el papel de un distribuidor previo a la portería.

Namespaces = mini-barrios (señalización y orden, no muros)

Un *Namespace* es una partición lógica del clúster: aporta *señalización* y *orden* para nombrar y agrupar recursos, pero no constituye una barrera física. Facilita organizar por equipos o entornos (dev/test/prod) y evitar colisiones de nombres sin multiplicar clústeres [2].
La señalización implica unicidad de nombres dentro del namespace. Así, pueden coexistir dos servicios llamados api si residen en mini-barrios distintos (p. ej., api en dev y api en prod) [2].
El nombre DNS de un servicio incluye el namespace: <servicio>.<namespace>.svc.cluster.local. Dentro del mismo mini-barrio se admite el nombre corto (api); para acceder a otro, es preferible el FQDN (api.prod.svc.cluster.local) para evitar ambigüedades y asegurar enrutamiento predecible [13], [2].
Por defecto, los namespaces no aíslan el tráfico: la conectividad *east–west* permanece abierta. Para definir *quién puede hablar con quién*, se emplean *NetworkPolicies* (si el CNI las soporta), especificadas con *labels* y puertos. Este enfoque se alinea con *Zero Trust*: permitir sólo lo necesario y aplicar verificación explícita [4], [24], [25], [26].

Recorrido completo (resumen operativo)

1. **Cliente externo** → (opcional) **balanceador → portería (Ingress/Gateway)** [17].
2. La portería aplica reglas L7 y dirige la solicitud a un **Service** (central) [3], [13].
3. El Service consulta sus **EndpointSlices** (inventario de réplicas activas) y selecciona un Pod [1].
4. El tráfico circula por las **calles CNI** hasta la **casa (Pod)** en el **barrio (Node)** [2], [4], [18].

Por último, la analogía de ciudad permite articular el *K8s Networking*: Pods como viviendas (identidad IP efímera), Nodes como barrios (recursos compartidos), Services como central estable (balanceo con *EndpointSlices*), Namespaces como mini-barrios para organización, e Ingress/Gateway como portería (gobierno de entrada). Todo opera sobre la malla vial provista por CNI, que entrega la conectividad fundamental del clúster [1]–[4], [13], [17], [18], [20], [21].

CNI

¿Qué es CNI y qué resuelve?

El *Container Network Interface* (CNI) es la especificación y el conjunto de *plugins* que habilitan la conectividad de red de los Pods en Kubernetes. En términos prácticos, CNI define cómo el *runtime* de contenedores solicita a un plugin que "conecte" un Pod a la red: creación y unión de interfaces, asignación de direcciones IP mediante IPAM, configuración de rutas y, cuando corresponde, reglas de traducción/puertos en el *host* [18], [4], [21], [20].
El ciclo de vida se basa en operaciones simples (ADD/DEL) encadenables, lo que permite combinar responsabilidades (p. ej., un plugin para IPAM y otro para mapeo de puertos) [18], [21], [20].

Funciones núcleo

- **Identidad de red del Pod.** Otorga al Pod su propia IP (modelo *IP-por-Pod*) y asegura la reachabilidad *east–west* en el clúster [2].
- **Conexión y enrutamiento.** Añade interfaces virtuales, instala rutas y coordina con el *host* el paso del tráfico hacia/desde cada Pod [18], [4].
- **Gestión de direcciones (IPAM).** Reserva/entrega direcciones desde rangos predefinidos (ejemplo: *host-local*) y evita colisiones [20], [18].

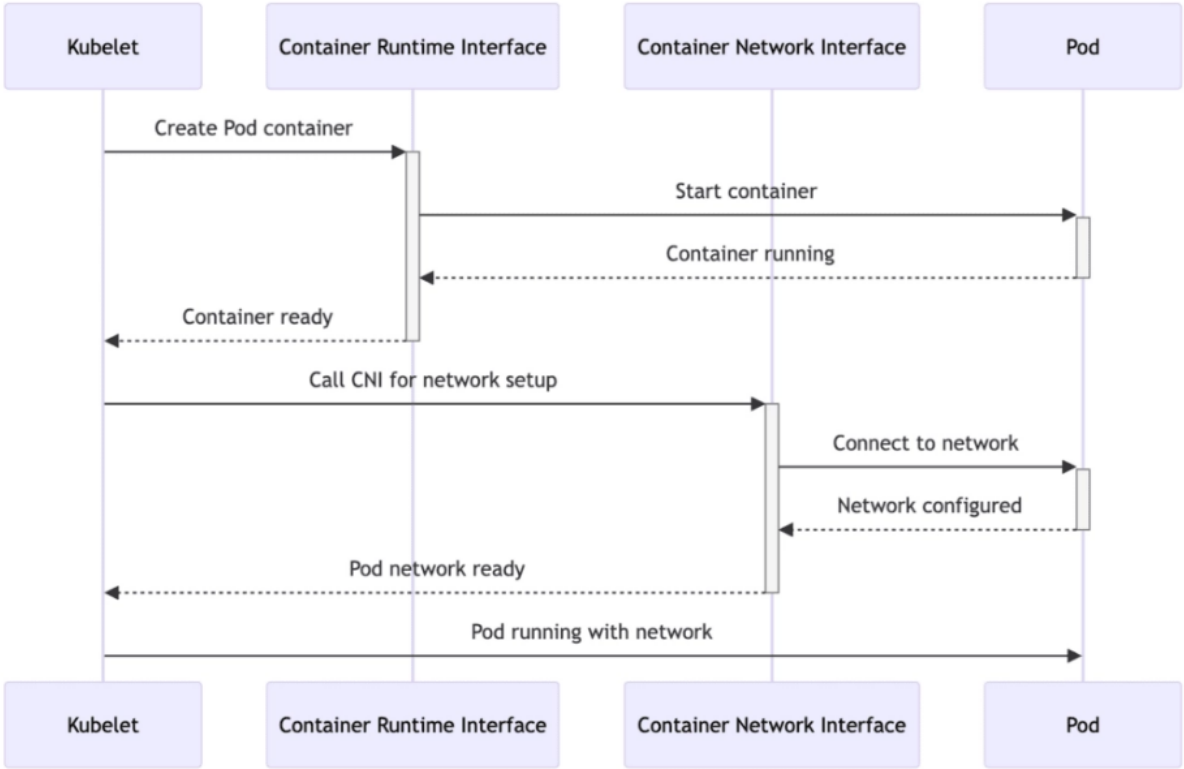


Figura 2. *Workflow* del Container Network Interface (CNI) mostrando las interacciones entre Kubelet, Pod y plugins CNI.

¿Por qué importa?

La elección de CNI (y su *dataplane*) condiciona latencia, *throughput* y consumo de CPU, con efectos directos sobre el sobredimensionamiento: estudios comparativos en *edge*/nube muestran variaciones relevantes entre modos (túnel vs. ruteo nativo) y aceleraciones (*fast paths*) [7], [8], [11], [22]. Además, la sintonía de parámetros como la MTU —clave en entornos con encapsulación— evita fragmentación y pérdidas de rendimiento [12]. En conjunto, un CNI bien seleccionado y calibrado facilita automatizar políticas, sostener SLOs y reducir recursos ociosos [4], [7], [8], [12].

Dataplane: overlay vs. ruteo directo

En Kubernetes, el *dataplane* de red que implementa un CNI puede seguir dos enfoques predominantes:

- **Overlay.** Encapsula el tráfico entre nodos (p. ej., mediante VXLAN sobre UDP) para transportar paquetes de Pods sin requerir cambios en la red física. Ventajas: simplicidad operativa y desacoplamiento del direccionamiento del clúster respecto al *underlay*. Costes: cabeceras adicionales, posibles ajustes de MTU y cierta carga de CPU por encapsulación/descapsulación.
- **Ruteo nativo (ruteo directo).** Los Pods son alcanzables a través de rutas L3 entre nodos sin túneles; suele ofrecer menor latencia y mejor aprovechamiento de CPU, pero exige integración clara con la red subyacente y un plan de direccionamiento coherente.

Estudios comparativos en entornos *edge* y nube muestran diferencias medibles de latencia, *throughput* y uso de CPU entre CNIs y modos de operación, lo que refuerza la importancia de seleccionar el *dataplane* según el contexto de despliegue [7], [8]. Adicionalmente, la implementación concreta del *fastpath* (p. ej., conmutación basada en OVS o programas eBPF) condiciona el rendimiento y la observabilidad del plano de datos [11], [22].

Asignación de direcciones (IPAM)

La identidad de red del Pod se materializa con una dirección IP propia, asignada por el mecanismo de *IP Address Management* (IPAM) del CNI. En líneas generales, existen estrategias por nodo (cada nodo administra un rango local) o globales (un *pool* común), con implicaciones distintas en simplicidad operativa y uso de direcciones. El plugin *host-local* ilustra un enfoque minimalista: asigna direcciones desde ficheros locales por nodo, garantizando unicidad sin dependencias externas. Un plan de direccionamiento bien definido (rangos, *leases*, reutilización y limpieza) minimiza colisiones, facilita *dual-stack* y reduce incidencias durante escalado o rotación de Pods [18], [20].

MTU y fragmentación en redes con túneles

La encapsulación propia de los *overlays* reduce la MTU efectiva: parte de los bytes del paquete se consumen en cabeceras adicionales. Si la MTU no se ajusta, pueden aparecer fragmentación, caídas de *throughput* y aumentos de latencia. Las guías prácticas recomiendan calcular la MTU efectiva restando el *overhead* del túnel y validar el ajuste mediante pruebas controladas; este procedimiento es especialmente relevante en despliegues con VXLAN o cifrado a nivel de red [12].

En síntesis, un ajuste conservador de MTU, verificado empíricamente, suele prevenir pérdidas de rendimiento en clústeres con encapsulación.

Network MTU	Calico MTU	Calico MTU with IP-in-IP (IPv4)	Calico MTU with VXLAN (IPv4)	Calico MTU with VXLAN (IPv6)	Calico MTU with WireGuard (IPv4)	Calico MTU with WireGuard (IPv6)
1500	1500	1480	1450	1430	1440	1420
9000	9000	8980	8950	8930	8940	8920
1500 (AKS)	1500	1480	1450	1430	1340	1320
1460 (GCE)	1460	1440	1410	1390	1400	1380
9001 (AWS Jumbo)	9001	8981	8951	8931	8941	8921
1450 (OpenStack VXLAN)	1450	1430	1400	1380	1390	1370

Figura 3. Ajuste de MTU en presencia de encapsulación (p. ej., VXLAN) para evitar fragmentación y pérdida de rendimiento [12].

Cómo circula el tráfico hacia los Services

Aunque CNI proporciona conectividad entre Pods y nodos, el direccionamiento estable para el consumo de aplicaciones lo otorgan los *Services* de Kubernetes. Un Service expone una Dirección IP Virtual – *VIP* (p. ej., *ClusterIP*) y se apoya en estructuras como *EndpointSlices* para conocer qué Pods saludables pueden recibir tráfico. El CNI asegura que, una vez resuelta la VIP, el paquete alcance el Pod correspondiente a través de las rutas del clúster; el modo de exposición (ClusterIP/NodePort/LoadBalancer) define cómo entra y se distribuye el tráfico desde dentro o fuera del clúster. Comprender esta separación de responsabilidades (CNI = conectividad/IP; Service = *VIP* y selección de destinos) es clave para razonar sobre fallos y cuellos de botella [3], [13], [1], [2].

Observabilidad sencilla a nivel CNI

Para operar y optimizar una red de clúster es útil contar con visibilidad de "quién habla con quién", volúmenes de tráfico y patrones temporales. Soluciones de *flow visibility* a nivel CNI permiten recolectar y agregar registros de flujo (origen, destino, puertos, bytes/paquetes) y correlacionarlos con identidades lógicas (namespaces, *labels*). Esta información facilita detectar rutas calientes, validar políticas y priorizar mejoras. En el ámbito de microservicios, la observabilidad de red complementa métricas de aplicación para diagnosticar lazos críticos o dependencias inesperadas [23], [6].

Rendimiento: qué medir y qué esperar

La evaluación del CNI debería incluir latencia (promedio y colas P95/P99), *throughput* sostenido y coste de CPU por paquete/conexión. También conviene fijar el modo del *dataplane* y el perfil de tráfico (tamaño de paquete, concurrencia, mezcla entrada/salida) para obtener comparaciones reproducibles. La bibliografía reporta patrones consistentes entre modos (overlay vs. ruteo directo) y entre *fast-paths* (p. ej., OVS o eBPF), con variaciones significativas en entornos con recursos limitados o altas tasas de conexión [7], [8]. Estas métricas orientan decisiones de diseño y *tuning* para cumplir SLOs de latencia y capacidad.

Evitar sobredimensionamiento vía CNI

Un CNI bien seleccionado y sintonizado ayuda a reducir recursos "por si acaso". Tres prácticas son especialmente efectivas:

1. elegir el *dataplane* según el entorno (overlay para simplicidad o ruteo directo cuando se prioriza latencia/CPU);
2. ajustar MTU y validar que no haya fragmentación;
3. automatizar *NetworkPolicies* en clave de *Zero Trust* para limitar tráfico innecesario y reducir trabajo de red en el plano de datos.

Complementar estas decisiones con visibilidad de flujos permite cerrar el ciclo de mejora continua y sostener SLOs con menos CPU/memoria [7], [8].

Políticas de red en Kubernetes

En Kubernetes, la red del clúster está pensada para que los Pods puedan comunicarse entre sí de forma directa desde su creación. Este comportamiento "abierto por defecto" simplifica el despliegue inicial, pero plantea retos cuando existen múltiples aplicaciones, equipos y niveles de criticidad. Las **políticas de red** (*NetworkPolicies*) permiten definir, de forma declarativa, **qué flujos de comunicación están permitidos** y, por exclusión, cuáles deben bloquearse. Estas políticas se apoyan en la

conectividad básica provista por el complemento de red (**CNI**) y conviven con los mecanismos de descubrimiento y balanceo que ofrecen los **Services** y sus **EndpointSlices** [2].

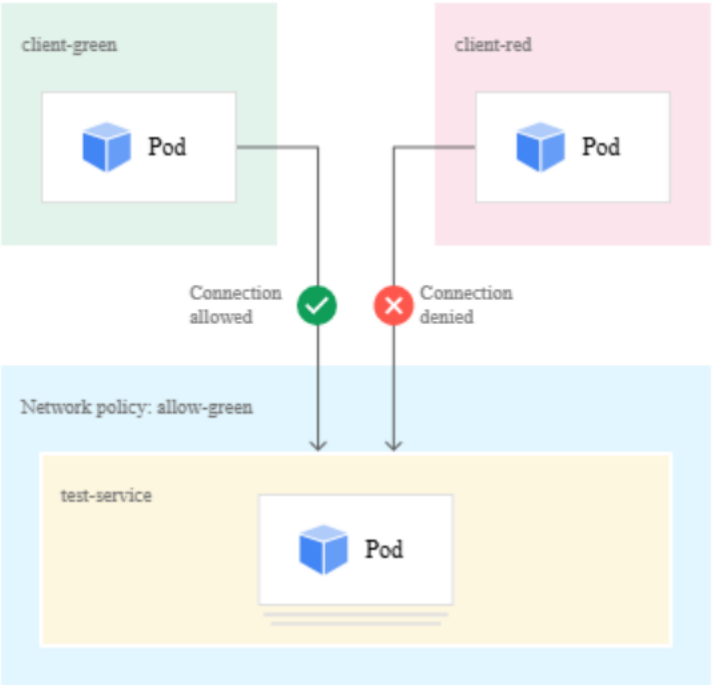


Figura 4. Representación de política de red en Kubernetes (NetworkPolicy) con control de conexiones permitidas y denegadas. Fuente: elaboración propia.

Ubicación en la arquitectura (qué hace cada capa)

- **Conectividad (CNI).** El CNI asigna IP a los Pods, crea las interfaces y establece rutas. Para que las políticas de red surtan efecto, el CNI elegido **debe soportar su aplicación**; si no lo hace, la política se aceptará, pero no se hará cumplir en el plano de datos [4], [18].
- **Servicio estable (Service + VIP).** Los Services proporcionan una **dirección IP virtual (VIP)** que permite llegar a un conjunto de Pods equivalentes; kube-proxy mantiene las reglas para redirigir el tráfico a las réplicas saludables según la información publicada en **EndpointSlices** [13], [3], [1].
- **Control de permisos (NetworkPolicy).** Las políticas regulan **si** un flujo está autorizado (entrada/salida, orígenes/destinos, puertos). No reemplazan a los Services ni al CNI: **los complementan** [2], [4], [13].

En una frase: **CNI = conectividad/IP**, **Service = VIP y réplicas**, **NetworkPolicy = permisos de tráfico** [2], [13], [4].

Conceptos operativos clave

Una **NetworkPolicy** se aplica a un grupo de Pods (elegidos por etiquetas o por *namespace*) y define:

- **Dirección:** entradas (*ingress*), salidas (*egress*) o ambas.
- **Ámbito:** quién puede hablar con quién (Pods, *namespaces*, rangos de IP) y por **qué puertos/protocolos**. El resultado es pasar de "todos se comunican con todos" a un **modelo de mínimo privilegio**, alineado con los principios de **Zero Trust** (no confiar por defecto; verificar y autorizar explícitamente) [24], [25]. La documentación de red de Kubernetes y de complementos CNI recoge estos comportamientos y requisitos de compatibilidad [2], [4], [18].

Buenas prácticas de diseño

1. **Etiquetado consistente.** Definir etiquetas simples y estables para seleccionar Pods y *namespaces* (p. ej., tier=frontend|api|db, env=dev|prod) facilita escribir políticas claras y auditables [2].
2. **Cerrar por defecto, abrir lo necesario.** Empezar con políticas de **denegación por defecto** (ingress y, si procede, egress) por *namespace*, y después añadir reglas de *permitir* para cada flujo requerido [2].
3. **Separar responsabilidades.** Los Services aportan estabilidad y balanceo (VIP + kube-proxy + EndpointSlices); las políticas deciden **si** ese tráfico está autorizado. Mantener esta separación simplifica el diagnóstico de incidencias.
4. **Visibilidad de flujos.** Activar **flow visibility** (p. ej., exportación IPFIX y cuadros de mando) ayuda a verificar "quién habla con quién", a validar políticas y a descubrir dependencias no documentadas.

Observabilidad y cumplimiento

En arquitecturas de microservicios, el número de dependencias crece rápidamente. La literatura reciente sistematiza los retos y prácticas de **observabilidad** (métricas, trazas y, especialmente aquí, **flujos de red**) para garantizar que la seguridad no degrade la capacidad de diagnóstico [6]. En Kubernetes, soluciones de **network-flow visibility** a nivel CNI (p. ej., Antrea con IPFIX)

	permiten correlacionar flujos con identidades lógicas (Pods, *namespaces*, etiquetas) y demostrar el cumplimiento de las políticas aplicadas. **Rendimiento: qué medir y cómo interpretarlo** Las políticas de red se evalúan sobre el **plano de datos** del CNI; por ello, el impacto en **latencia, *throughput* y CPU** depende de la implementación del complemento y del modo de red (túneles *overlay* frente a **ruteo directo**) [7], [8]. Estudios comparativos y trabajos sobre *dataplanes* (como Open vSwitch, OVS) muestran **diferencias medibles** entre enfoques y optimizaciones, especialmente en entornos *edge* o de alta concurrencia [7], [8]. En escenarios con encapsulación (p. ej., VXLAN), el ajuste de **MTU** evita fragmentación y pérdidas de rendimiento, por lo que conviene considerarlo cuando se endurezcan las políticas y cambie el patrón de flujos [12]. En el extremo, técnicas basadas en **eBPF** añaden capacidades de filtrado/observación en el kernel con implicaciones de eficiencia y respuesta ante intrusiones [27]. **Relación con otros componentes de seguridad** Las políticas de red operan en **capa 3/4** (IPs y puertos). Para necesidades de **capa 7** (HTTP/gRPC, identidades de servicio, *mTLS*, *rate-limiting*), la literatura recomienda considerar **service mesh** y/o **API gateways** como capas complementarias (y no excluyentes). Como línea estratégica, el enfoque **Zero Trust** del NIST sirve de referencia para ordenar controles y adoptar el principio de **mínimo privilegio** de forma progresiva y verificable. **Riesgos y precauciones habituales** • **Compatibilidad del CNI.** Verificar que el complemento **implementa NetworkPolicy**; de lo contrario, las reglas declaradas no se harán cumplir . • **Egresos olvidados.** Aislar solo entradas y dejar salidas abiertas puede permitir conexiones no deseadas a Internet u otros dominios; si el caso lo requiere, definir **egress** explícito [2]. • **Confusiones por NAT/camino de entrada.** En exposiciones vía *LoadBalancer* o *NodePort*, la IP de origen que ve el Pod puede ser la del balanceador/nodo, no la del cliente final (según configuración), lo que afecta reglas basadas en IP. • **Segmentación de capa 2 ≠ políticas de capa 3/4.** La separación por VLAN (IEEE 802.1Q) no sustituye el control lógico entre Pods; ambos enfoques pueden coexistir con objetivos distintos [19]. Las políticas de red son el **mecanismo declarativo** que lleva a la red de Kubernetes el principio de **mínimo privilegio**. Bien diseñadas, permiten limitar superficies de exposición, ordenar la comunicación entre servicios y mejorar la trazabilidad del tráfico. Su efectividad descansa en tres pilares: (i) un **CNI compatible y correctamente ajustado**; (ii) un **catálogo de dependencias** sustentado en **visibilidad de flujos**; y (iii) una **operación disciplinada** (cerrar por defecto, abrir solo lo imprescindible, medir y ajustar). Combinadas con la capa de servicio (VIP + EndpointSlices) y, cuando aplique, con capas L7 (service mesh/gateway), constituyen una base sólida para elevar la seguridad sin perder claridad operativa ni capacidad de diagnóstico .
Solución Tecnológica	Se desarrollará un modelo de trabajo y un conjunto de herramientas para la evaluación y configuración de CNI en Kubernetes, que permitan a las organizaciones seleccionar y configurar de manera más objetiva los plugins más apropiados para sus necesidades específicas. Esta solución busca reducir la dependencia de decisiones basadas en preferencias personales o información incompleta, proporcionando criterios técnicos medibles y reproducibles. La solución se compone de cuatro elementos principales que trabajan de manera integrada para proporcionar una experiencia completa de evaluación y despliegue. Esta implementación estratégica permitirá a las empresas migrar de configuraciones basadas en prueba y error a un enfoque sistemático y basado en evidencia, generando mayor eficiencia operacional y reducción de incidentes de producción. **Componentes Específicos a Desarrollar** **1. Guía metodológica de evaluación del modelo** • Procedimientos paso a paso para instalar y probar cada CNI • Metodología de pruebas de rendimiento y seguridad • Criterios de medición específicos y reproducibles **Entregable**: Documento digital (PDF) con lista de verificación **2. Kit de Pruebas Automatizado** • Scripts de instalación y configuración automatizada • Herramientas de medición de rendimiento integradas • Escenarios de prueba definidos y reproducibles. **Entregable**: Paquete de scripts con documentación de uso

	3. Sistema de Puntuación Comparativa • Matriz de evaluación con criterios ponderados • Calculadora automática de recomendaciones • Plantilla adaptable a diferentes contextos organizacionales **Entregable**: Hoja de cálculo parametrizable (Excel/CSV) **4. Repositorio de Configuraciones Base** • Configuraciones de seguridad predefinidas para cada tecnología • Plantillas de políticas de red de referencia ajustables a cada contexto • Documentación de implementación **Entregable**: Conjunto de archivos de configuración organizados por tecnología Al finalizar el proyecto, las organizaciones contarán con herramientas concretas que les permitirán tomar decisiones informadas sobre tecnologías de red para Kubernetes, reducir el tiempo de evaluación e implementar configuraciones probadas y seguras.
Metodología	**Scrum** es un método para manejar proyectos que se basa en dar pequeñas partes de trabajos y tener mucha cooperación entre los miembros del grupo. Esta forma de trabajo viene del manejo rápido de software, aunque su uso ha ido hacia varios ámbitos laborales. Lo clave de este modelo es dividir proyectos difíciles en ciclos de trabajo llamados "sprints", que suelen ser desde una a cuatro semanas. En cada sprint, el equipo se enfoca en hacer un grupo claro de tareas que fueron separadas y puestas en orden. Los roles importantes dentro de esta metodología incluyen al Product Owner, quien actúa como representante de lo que requiere el cliente y define las características del producto, el Scrum Master, que facilita el proceso y elimina obstáculos y el Equipo de Desarrollo, responsable de la creación del producto final. Scrum fue seleccionado como marco de trabajo debido a que la naturaleza del proyecto exige resultados tangibles en periodos cortos de tiempo y una constante capacidad de ajuste. Al comparar varios CNI (Container Network Interface) en Kubernetes, el equipo debe, en cuestión de semanas, instalar, medir, contrastar y refinar métricas tanto de rendimiento como de seguridad. Este ciclo de "prueba → análisis → mejora" se adapta de forma natural a los sprints de Scrum, donde en cada iteración se entrega un incremento funcional (por ejemplo, un script de pruebas o un panel comparativo) y, al finalizar el sprint, se evalúan datos reales que sirven como base para planear los siguientes pasos. El proyecto es desarrollado por un equipo pequeño, compuesto por dos Desarrolladores, un Product Owner de perfil académico (el director del trabajo) y uno o varios tutores técnicos que asumen el rol de Scrum Master, encargados de remover impedimentos y validar las buenas prácticas. Con una estructura de equipo reducida, las ceremonias breves propias de Scrum (como la reunión diaria de diez minutos, la revisión y la retrospectiva al final de cada sprint) permiten mantener informados a los asesores sin sobrecargarlos con documentación innecesaria. Un aspecto clave que motivó esta elección es la incertidumbre técnica presente en el proyecto. Al no saber desde el inicio qué CNI tendrá un mejor rendimiento bajo ciertas condiciones de carga ni qué métricas adicionales será necesario instrumentar, Scrum ofrece la flexibilidad para ajustar el Product Backlog en cada sprint. Por ejemplo, si se descubre que el consumo energético es más relevante que la latencia, es posible reorganizar las tareas y priorizar nuevas mediciones sin necesidad de rehacer toda la planificación.
Cronograma	**ANEXO 1** El cronograma del proyecto se establece con una fecha de inicio en julio de 2025 y una finalización en febrero de 2026, contemplando una duración total de ocho meses. Para consultar información detallada sobre las fases específicas del proyecto, se recomienda revisar el Anexo 1.
Presupuesto y fuentes de financiación	**ANEXO 2**
Bibliografía de VÍNCULOS	**[1]** D. F. Solano and M. P. Gutiérrez, "Algoritmos de aprendizaje automático para optimizar las redes 5G: Desarrollo y evaluación del rendimiento," *Rev. Vínculos*, vol. 20, no. 1, Apr. 2023. **[2]** M. I. Cordero-Pérez and P. E. Salas-Duarte, "Inteligencia Artificial en la Gestión de Redes Telemáticas: Avances, Tendencias y Aplicaciones Actuales," *Rev. Vínculos*, vol. 21, no. 1, May 2024.

[3]
M. Peña-Robayo and A. del P. Moreno, "Amenazas emergentes en la computación en la nube: desafíos de seguridad y respuesta," *Rev. Vínculos*, vol. 21, no. 1, Apr. 2024.
[4]
J. A. Romero-Pinto, C. E. Herrera-Carvajal, and M. C. Puerta-Gómez, "Optimización del consumo energético en redes 5G utilizando un enfoque híbrido de algoritmos ACO y BA basado en SDN," *Rev. Vínculos*, vol. 20, no. 1, Aug. 2023.
[5]
E. Villanueva, "Seguridad en Redes Ad Hoc: Implementación de Protocolos de Comunicación Usando Criptografía de Clave Pública," *Rev. Vínculos*, vol. 21, no. 1, Mar. 2024.

Bibliografía de TECNURA

[1]
J. Castellanos and C. G. Castrillón Arias, "Diseño de una ruta metodológica para la toma de decisiones en la adquisición de software," *Tecnura*, vol. 27, no. 75, pp. 38–50, Jan. 2023.
[2]
J. C. Carreño Pérez and A. Espinel Ortega, "Identificación de activos y ciber activos críticos en sistemas de transmisión de energía eléctrica," *Tecnura*, vol. 24, no. 65, pp. 27–38, Jul. 2020.
[3]
D. A. Giral Ramírez, J. A. Muñoz Romero, and C. A. Ramírez Vanegas, "Monitoring system for electrical variables implementing blockchain and Python," *Tecnura*, vol. 26, no. 71, pp. 16–29, Jan. 2022.
[4]
C. G. Hidalgo Suárez, V. A. Bucheli-Guerrero, and H. A. Ordóñez-Eraso, "Artificial Intelligence and Computer-Supported Collaborative Learning in Programming: A Systematic Mapping Study," *Tecnura*, vol. 27, no. 75, pp. 175–206, Jan. 2023.
[5]
L. A. Luengas and L. F. Wanumen Silva, "Modelos computacionales en la posturografía," *Tecnura*, vol. 26, no. 73, pp. 30–48, Jul. 2022.

Bibliografía

[1] "EndpointSlices", *Kubernetes*. [En línea]. Disponible en: https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/. [Consultado: 21-ago-2025].
[2] "Cluster networking", *Kubernetes*. [En línea]. Disponible en: https://kubernetes.io/docs/concepts/cluster-administration/networking/. [Consultado: 21-ago-2025].
[3] "Virtual IPs and Service proxies", *Kubernetes*. [En línea]. Disponible en: https://kubernetes.io/docs/reference/networking/virtual-ips/. [Consultado: 21-ago-2025].
[4] "Network plugins", *Kubernetes*. [En línea]. Disponible en: https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/. [Consultado: 21-ago-2025].
[5] D. E. Eisenbud *et al.*, "Maglev: A fast and reliable software network load balancer", *Usenix.org*. [En línea]. Disponible en: https://www.usenix.org/sites/default/files/nsdi16-paper-eisenbud.pdf. [Consultado: 21-ago-2025].
[6] F. Gomes, P. Rego, y F. Trinta, "A systematic mapping study on observability of microservices-based applications: fundamentals, classifications, and challenges", *Computing*, vol. 107, núm. 9, 2025.
[7] Z. Kang, K. An, A. Gokhale, y P. Pazandak, "A comprehensive performance evaluation of different kubernetes CNI plugins for edge-based and containerized publish/subscribe applications", *Vanderbilt.edu*. [En línea]. Disponible en: https://www.dre.vanderbilt.edu/~gokhale/WWW/papers/IC2E21_CNI_Eval.pdf. [Consultado: 21-ago-2025].
[8] G. Koukis, S. Skaperas, I. A. Kapetanidou, L. Mamatas, y V. Tsaoussidis, "Performance evaluation of kubernetes networking approaches across constraint edge environments", *arXiv [cs.NI]*, 2024.
[9] *Mdpi.com*. [En línea]. Disponible en: https://www.mdpi.com/2079-9292/13/19/3972. [Consultado: 21-ago-2025].
[10] best practices, "Elevating Kubernetes network security through Cilium deployment", *Fh-joanneum.at*. [En línea]. Disponible en: https://epub.fh-joanneum.at/obvfhjhs/download/pdf/11499653. [Consultado: 21-ago-2025].
[11] W. Tu, Y.-H. Wei, G. Antichi, y B. Pfaff, "revisiting the open vSwitch dataplane ten years later", en *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, 2021.
[12] "Configure MTU to maximize network performance", *Tigera.io*. [En línea]. Disponible en: https://docs.tigera.io/calico/latest/networking/configuring/mtu. [Consultado: 21-ago-2025].
[13] "Service", *Kubernetes*. [En línea]. Disponible en: https://kubernetes.io/docs/concepts/services-networking/service/. [Consultado: 21-ago-2025].
[14] *Polito.it*. [En línea]. Disponible en: https://webthesis.biblio.polito.it/28635/1/tesi.pdf. [Consultado: 21-ago-2025].
[15] Master's thesis, "Kubernetes networking: Comparative insights into API gateways and service mesh implementations", *Aalto.fi*. [En línea]. Disponible en: https://aaltodoc.aalto.fi/bitstreams/000efe1b-5f19-494c-a8c7-3fdb1ac62e74/download. [Consultado: 21-ago-2025].
[16] M. Kotenko, D. Moskalyk, V. Kovach, y V. Osadchyi, "Navigating the challenges and best practices in securing microservices architecture", *Ceur-ws.org*. [En línea]. Disponible en: https://ceur-ws.org/Vol-3826/paper1.pdf. [Consultado: 21-ago-2025].
[17] "Roles and personas - kubernetes gateway API", *K8s.io*. [En línea]. Disponible en: https://gateway-api.sigs.k8s.io/concepts/roles-and-personas/. [Consultado: 21-ago-2025].

[18] "CNI", *Cni.dev*. [En línea]. Disponible en: https://www.cni.dev/docs/spec/. [Consultado: 21-ago-2025].

[19] *Ieee.org*. [En línea]. Disponible en: https://standards.ieee.org/ieee/802.1Q/6844/. [Consultado: 21-ago-2025].

[20] "host-local IP address management plugin", *Cni.dev*. [En línea]. Disponible en: https://www.cni.dev/plugins/current/ipam/host-local/. [Consultado: 21-ago-2025].

[21] "Port-mapping plugin", *Cni.dev*. [En línea]. Disponible en: https://www.cni.dev/plugins/current/meta/portmap/. [Consultado: 21-ago-2025].

[22] B. Pfaff *et al.*, "The design and implementation of open vSwitch", *Usenix.org*. [En línea]. Disponible en: https://www.usenix.org/system/files/conference/nsdi15/nsdi15-paper-pfaff.pdf. [Consultado: 21-ago-2025].

[23] "Antrea", *Antrea.io*. [En línea]. Disponible en: https://antrea.io/docs/v1.0.0/docs/network-flow-visibility/. [Consultado: 21-ago-2025].

[24] S. Rose, O. Borchert, S. Mitchell, y S. Connelly, "Zero Trust Architecture", National Institute of Standards and Technology, 2020.

[25] O. Borchert, G. Howell, A. Kerman, S. Rose, y M. Souppaya, "Implementing a zero trust architecture :: High-level document", National Institute of Standards and Technology (U.S.), Gaithersburg, MD, 2025.

[26] R. Chandramouli y Z. Butcher, "Building secure microservices-based applications using service-mesh architecture", National Institute of Standards and Technology, Gaithersburg, MD, 2020.

[27] J. Zhang, P. Chen, Z. He, H. Chen, y X. Li, "Real-time intrusion detection and prevention with neural network in kernel using eBPF", en *2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2024, pp. 416–428.

Acta de compromiso sobre propiedad intelectual	**ANEXO 3**

EVALUACION DEL DIRECTOR (Marcar con X solo un ítem. Para todas las modalidades)	
- ***Aprobado***. Significa que el trabajo puede comenzar, ya que está claro su planteamiento y formulación.	
- ***Rechazado*.** Significa que, por alguna razón, debidamente justificada por el evaluador, el trabajo no reúne losrequisitos solicitados para ser un trabajo de grado.	
- ***Modificar*.** Significa que el estudiante debe hacer algunos ajustes a su propuesta, los cuales deberán ser discutidos y concertados con el tutor o director del trabajo de grado y posteriormente presentados de nuevo ante el Consejo Curricular respectivo para su revisión.	
Comentarios:	

Atentamente informo acerca de mi intención de participar como Director en el desarrollo del trabajo de Grado OPTIMIZACIÓN INTEGRAL DE REDES KUBERNETES: COMPARATIVA DE CNIS, AUTOMATIZACIÓN DE NETWORK POLICIES Y REDUCCIÓN DE SOBREDIMENSIONAMIENTOS que será desarrollando por los estudiantes HOLMAN AUDREY ALBA CASTRO con CC.1007305494 y DUWAN ESTIVEN SIERRA GUERRERO con CC. 1033777208 Declaro además que, conozco el reglamento de trabajos de grado de la Facultad Tecnológica de laUniversidad Distrital Francisco José de Caldas y los deberes y derechos que como Director esto implica. El desarrollo del proyecto se hará de conformidad con lo estipulado en dicho acuerdo.

Artículo 11. Docente Director. Para toda modalidad de Trabajo de Grado, se designará un docente de carrera o docente de vinculación especial (Tiempo Completo Ocasional, Medio Tiempo Ocasional) en calidad de docente director, quién tendrá como principales funciones, las siguientes: **a.) Avalar la propuesta de trabajo de grado presentada por el o los estudiantes.** b.) Realizar el seguimiento del desarrollo del trabajo de grado propuesto. c.) Reportar la nota final a la coordinación del proyecto curricular antes de la semana 16 del periodo académico respectivo. d.) Asistir a la socialización y asignar la nota correspondiente del trabajo de grado de los estudiantes para el que ha sido designado como director. e.) Vigilar el cumplimiento del estatuto de propiedad intelectual establecido en el Acuerdo No. 04 de 2012 del Consejo Superior Universitario. f.) Firmar oportunamente el o las actas de evaluación de trabajo de grado.

GCASTMONT

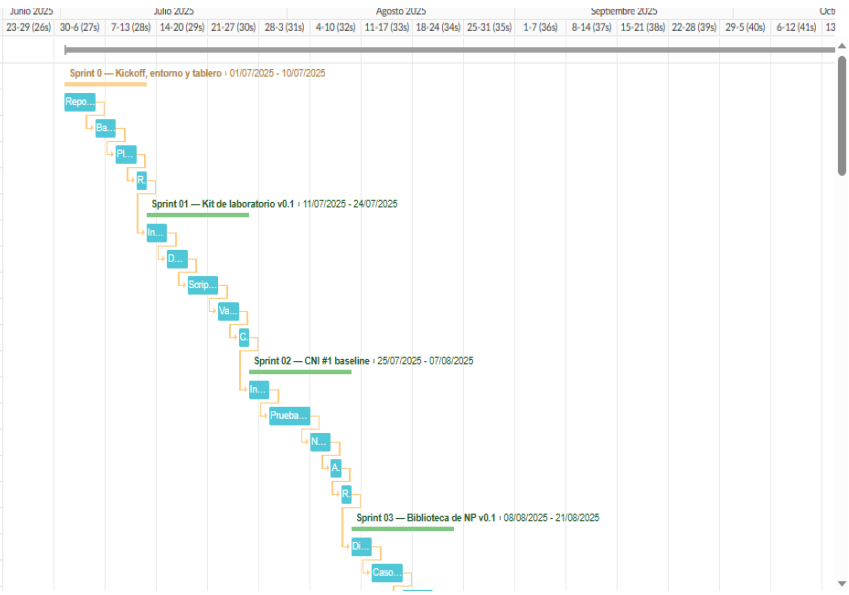
FIRMA (VISTO BUENO DEL DIRECTOR)
GERARDO ALBERTO CASTANG MONTIEL.

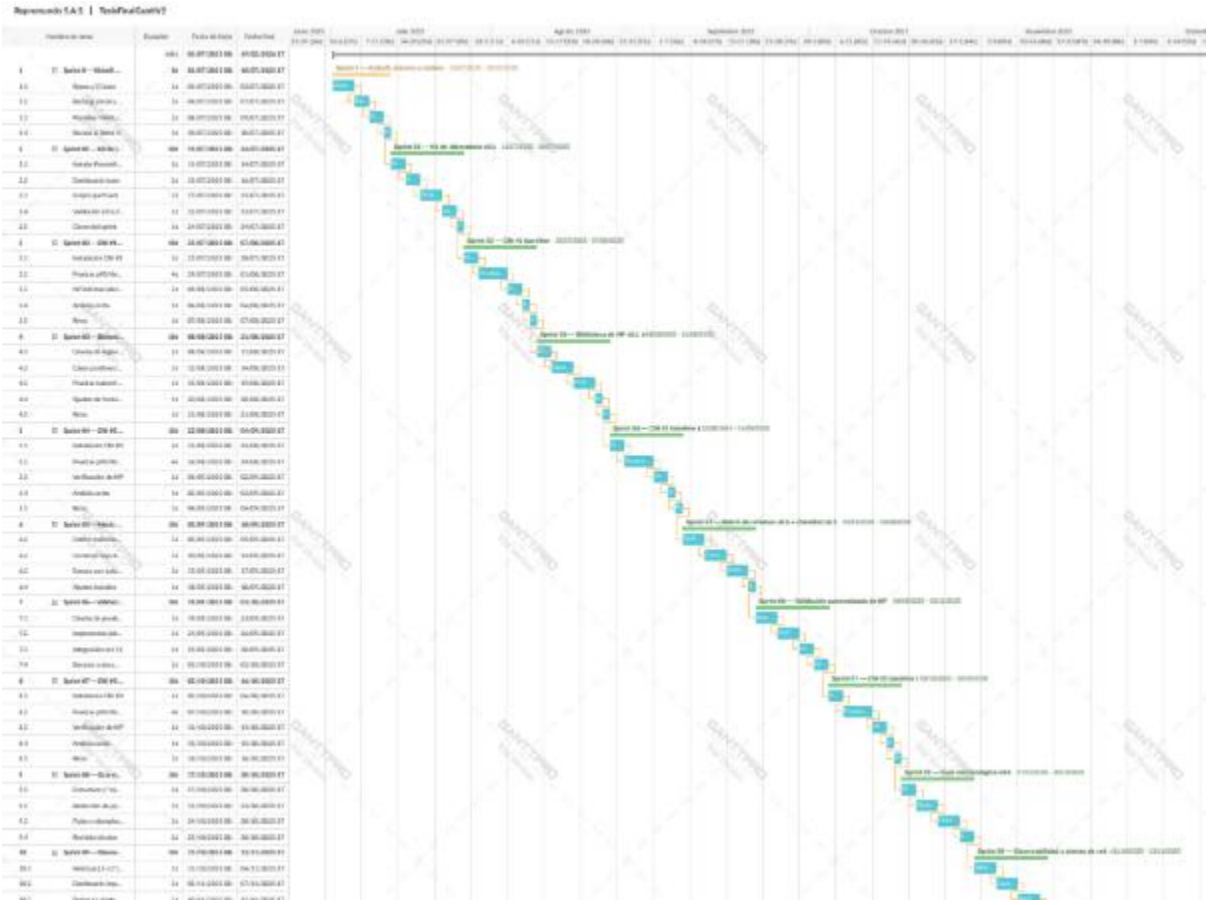
ANEXO 1

Para consultar información detallada relacionada con el Anexo 1. puede acceder al siguiente enlace de descarga:

https://drive.google.com/file/d/1t2pUpIBd8dQcV2DoxASKQxhyuNY-jYis/view?usp=sharing

	Nombre de tarea	Duración	Fecha de inicio	Fecha final
		168d	01/07/2025 08	19/02/2026 17
1	Sprint 0 – Kickoff, ...	8d	01/07/2025 08	10/07/2025 17
1.1	Repos y CI base	3d	01/07/2025 08:	03/07/2025 17
1.2	Backlog inicial y ...	2d	04/07/2025 08:	07/07/2025 17
1.3	Plantillas YAML/...	2d	08/07/2025 08:	09/07/2025 17
1.4	Review & Retro 0	1d	10/07/2025 08:	10/07/2025 17
2	Sprint 01 – Kit de l...	10d	11/07/2025 08	24/07/2025 17
2.1	Instalar Prometh...	2d	11/07/2025 08:	14/07/2025 17
2.2	Dashboards base	2d	15/07/2025 08:	16/07/2025 17
2.3	Scripts iperf/wrk	3d	17/07/2025 08:	21/07/2025 17
2.4	Validación intra/i...	2d	22/07/2025 08:	23/07/2025 17
2.5	Cierre del sprint	1d	24/07/2025 08:	24/07/2025 17
3	Sprint 02 – CNI #1 ...	10d	25/07/2025 08	07/08/2025 17
3.1	Instalación CNI #1	2d	25/07/2025 08:	28/07/2025 17
3.2	Pruebas p95/thr...	4d	29/07/2025 08:	01/08/2025 17
3.3	NP mínimas (den...	2d	04/08/2025 08:	05/08/2025 17
3.4	Análisis corto	1d	06/08/2025 08:	06/08/2025 17
3.5	Retro	1d	07/08/2025 08:	07/08/2025 17
4	Sprint 03 – Bibliote...	10d	08/08/2025 08	21/08/2025 17
4.1	Diseño de reglas...	2d	08/08/2025 08:	11/08/2025 17
4.2	Casos positivos/...	3d	12/08/2025 08:	14/08/2025 17





ANEXO 2

PRESUPUESTO Y FUENTES DE FINANCIACION

FACTIBILIDAD

Factibilidad Técnica

El proyecto constituye un paquete de entrega académica compuesto por cuatro bloques (guía metodológica, kit de laboratorio, matriz de criterios y biblioteca de NetworkPolicies).

Para producirlos se requiere únicamente infraestructura de laboratorio y software libre por lo cual son necesarios los siguientes recursos:

Tabla 1: Recursos factibilidad técnica

Recursos	Cantidad	Valor Unitario	Valor Total
Equipos de computo	2	$ 5.000.000	$10.000.000
Smartphone	2	$2.500.000	$5.000.000
Licencias y software	-	-	-
Nodos k8s*	2	$49.285	$483.900
Calico Open Source	1	$0	$0
Flannel IO	1	$0	$0
Cilium IO	1	$0	$0
Grafana	1	$0	$0
Prometheus	1	$0	$0
		Total:	**$15.483.900**

Nota: Los costos de licenciamiento no se incluyen en esta estimación, ya que las herramientas y plugins seleccionados operan con licencias gratuitas. Sin embargo, estos costos se considerarán en la valoración final del proyecto.
* Precios digital ocean, pueden variar de acuerdo al dolar https://www.digitalocean.com/pricing/kubernetes

Factibilidad operativa

Para asegurar la viabilidad y éxito del proyecto, se consideran varios aspectos clave de manera integral. La infraestructura tecnológica adecuada será evaluada y establecida para el desarrollo y funcionamiento óptimo del proyecto, seleccionando las herramientas, plataformas y sistemas que mejor se adapten a las necesidades específicas del proyecto.

La planificación financiera será crucial para identificar fuentes de financiamiento, presupuestar adecuadamente los costos de implementación y operación, y gestionar los recursos financieros de manera eficiente para evitar limitaciones o retrasos.

Factibilidad legal

- Todo el stack (Kubernetes, Minikube, Calico, Cilium, Flannel, Antrea, Prometheus, Grafana, iperf, wrk, nmap) se distribuye bajo licencias permisivas (Apache 2.0, MIT, BSD o GPL) que permiten uso académico sin restricciones.

- No se manipulan datos personales ni información sensible, por lo que no aplica Habeas Data.
- El proyecto se entregará con referencias de las licencias y avisos de copyright correspondientes.

Factibilidad económica

La inversión se concentra en horas dedicadas y un pequeño rubro de energía/internet; el resto son herramientas open-source.

Tabla 2: Factibilidad económica Recursos Humanos

Integrante	Valor Hora	Horas	Valor Total
Holman Audrey Alba Castro	$35.000	3 Horas diarias en 182 dias	$19.110.000
Duwan Estiven Sierra Guerrero	$35.000	3 Horas diarias en 182 dias	$19.110.000
Gerardo Alberto Castang Montiel	$90.000	25 Horas	$2.700.000
		Total:	**$40.920.000**

Tabla 3: Factibilidad económica por recursos

Recursos	Cantidad	Valor Unitario	Valor Total
Internet	6 meses (2 personas)	$80.000	$960.000
Energia	6 meses (2 personas)	$40.000	$480.000
		Total:	**$1.440.000**

Factibilidad económica costo total

Recursos	Valor Total
Humano	$40.920.000
Tecnológico	$15.483.900
Otros recursos	$1.440.000
Costos imprevistos (10%)	$5.784.390
Total:	**$63.628.290**

ANEXO 3

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS
FACULTAD TECNOLÓGICA
INGENIERÍA EN TELEMÁTICA

ACTA DE COMPROMISO SOBRE PROPIEDAD INTELECTUAL[1]
PRESENTACIÓN PROPUESTA TRABAJO DE GRADO

AÑO: 2025 PERIODO ACADÉMICO: 3

FECHA: 28 de julio de 2025
CARRERA:INGENIERIA EN TELEMATICA POR CICLOS PROPEDEUTICOS
CÓDIGO: 20231678018 ESTUDIANTE: HOLMAN AUDREY ALBA CASTRO
CÓDIGO: 20231678001 ESTUDIANTE: DUWAN ESTIVEN SIERRA GUERRERO

TÍTULO DEL TRABAJO DE GRADO: OPTIMIZACIÓN INTEGRAL DE REDES KUBERNETES: COMPARATIVA DE CNIS, AUTOMATIZACIÓN DE NETWORKPOLICIES Y REDUCCIÓN DE SOBREDIMENSIONAMIENTOS.

MODALIDAD DEL TRABAJO DE GRADO: MONOGRAFÍA

Al presentar la propuesta de trabajo de Grado para evaluación y aprobación por parte del Consejo Curricular, hago la siguiente declaración:

DECLARACIÓN:

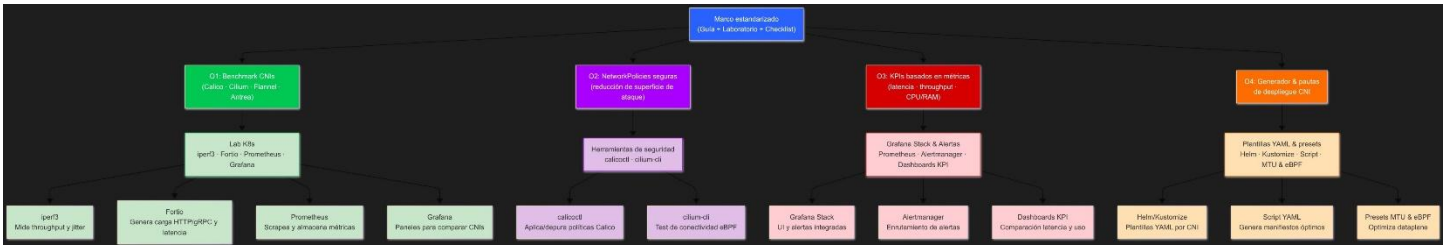
1 - Soy consciente que cualquier tipo de fraude en este trabajo es considerado como una falta grave en la Universidad. Al firmar, entregar y presentar esta propuesta de trabajo de Grado, doy expreso testimonio de que esta propuesta fue desarrollada de acuerdo con las normas establecidas por la Universidad. Del mismo modo, aseguro que no participé en ningún tipo de fraude y que en el trabajo se expresan debidamente los conceptos o ideas que son tomadas de otras fuentes.

2- Soy consciente de que el trabajo que realizaré incluirá ideas y conceptos del autor y el Director y podrá incluir material de cursos o trabajos anteriores realizados en la Universidad y por lo tanto, daré el crédito correspondiente y utilizaré este material de acuerdo con las normas de derechos de autor. Así mismo, no haré publicaciones, informes, artículos o presentaciones en congresos, seminarios o conferencias sin la revisión o autorización expresa del Director, quien representará en este caso a la Universidad.

HOLMAN AUDREY ALBA CASTRO
CC: 1007305494

DUWAN ESTIVEN SIERRA GUERRERO
CC: 1033777208

[1] Fuente: Universidad de los Andes, Bogotá. Departamento de Ingeniería Eléctrica y Electrónica.

ANEXO 4

Para consultar información detallada relacionada con el Anexo 4, puede acceder al siguiente enlace de descarga: https://drive.google.com/file/d/12pletSMaAnKDhLLZDZkmS_l8yISiPun2/view?usp=drive_link.